Task 10 - Practice Lab: Task Tracker Status Filter + PR Handoff

Handbook reference: Page 10 of 10 | Report date: 2026-10-13 | Folder: `task-10-practice-lab/`

1. Concept from the handbook

Exercise: add a status filter to the existing Task Tracker, preserve current behaviour, diagnose one relevant bug and prepare an evidence-backed PR handoff. Acceptance: All / Open / Done behave per criteria; a Done filter must never contain Open tasks. Complete means: diff reviewed, selected checks recorded, exact revision and limitations reported.

2. What was built

A complete small app: **task_tracker** package (filter + CLI + HTML render), a **baseline** copy of the pre-feature behaviour, **15 tests** (acceptance, negative, preserved behaviour, CLI), a **bug diagnosis** script and note, a generator for **PR_HANDOFF.md** from real results, and a **5-point rubric self-check** (Clarity, Scope, Correctness, Verification, Handoff).

3. Behaviours demonstrated

- All / Open / Done return the right tasks in any stored case
- Done never contains Open; Open and Done partition All
- Default list == baseline behaviour, order preserved, input not mutated
- Unknown status raises a clear error instead of an empty list
- Rubric self-check scores 5/5 from files on disk

4. Files in this task

`AGENTS.md`, `PR_HANDOFF.md`, `baseline/tracker_before.py`, `demo.py`, `diagnose_bug.py`, `docs/BUG_DIAGNOSIS.md`, `generate_handoff.py`, `render_ui.py`, `rubric_check.py`, `task_tracker/__init__.py`, `task_tracker/__main__.py`, `task_tracker/cli.py`, `task_tracker/data.json`, `task_tracker/tracker.py`, `tests/test_cli.py`, `tests/test_status_filter.py`, `ui/tracker_all.html`, `ui/tracker_done.html`, `ui/tracker_open.html`

Run it: `python demo.py` Test it: `python -m unittest discover -s tests`

5. Code screenshots

task_tracker/tracker.py

```python
"""Task Tracker core. Feature added in this lab: status filter (All / Open / Done)."""
import json
from pathlib import Path
from typing import Dict, List, Optional

STATUSES = ("all", "open", "done")
DATA_FILE = Path(__file__).with_name("data.json")


def load_tasks(path: Optional[Path] = None) -> List[Dict]:
    return json.loads(Path(path or DATA_FILE).read_text())


def filter_tasks(tasks: List[Dict], status: str = "all") -> List[Dict]:
    """Return the tasks matching `status` ('all' | 'open' | 'done', case-insensitive).

    * never mutates `tasks`, keeps original order
    * stored statuses may be 'open', 'Open', 'DONE'... so both sides are normalised
      (this is the bug diagnosed in docs/BUG_DIAGNOSIS.md)
    """
    wanted = (status or "all").strip().lower()
    if wanted not in STATUSES:
        raise ValueError(f"unknown status '{status}'. Choose from {', '.join(STATUSES)}")
    if wanted == "all":
        return list(tasks)
    return [t for t in tasks if t["status"].strip().lower() == wanted]


def list_tasks(tasks: List[Dict], status: str = "all") -> List[Dict]:
    """Existing entry point. Default argument keeps the OLD behaviour (show everything)."""
    return filter_tasks(tasks, status)


def render_table(tasks: List[Dict]) -> str:
    if not tasks:
        return "(no tasks)"
    rows = [f"{t['id']:>3}  {t['status'].lower():<5}  {t['title']}" for t in tasks]
    return "\n".join(["ID   STAT   TITLE", "---  -----  -----", *rows])
```

Main implementation: task_tracker/tracker.py

tests/test_status_filter.py

```python
import json
import sys
import unittest
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT))
sys.path.insert(0, str(ROOT / "baseline"))

from task_tracker import filter_tasks, list_tasks, load_tasks  # noqa: E402
import tracker_before  # noqa: E402

TASKS = load_tasks()
ids = lambda rows: [t["id"] for t in rows]


class AcceptanceCriteria(unittest.TestCase):
    """All / Open / Done behave as specified."""

    def test_all_returns_every_task(self):
        self.assertEqual(ids(filter_tasks(TASKS, "all")), [1, 2, 3, 4, 5, 6])

    def test_open_returns_open_tasks_any_case(self):
        self.assertEqual(ids(filter_tasks(TASKS, "open")), [1, 3, 5])

    def test_done_returns_done_tasks_any_case(self):
        self.assertEqual(ids(filter_tasks(TASKS, "done")), [2, 4, 6])

    def test_ui_labels_with_capital_letters_work(self):
        self.assertEqual(ids(filter_tasks(TASKS, "Done")), [2, 4, 6])
        self.assertEqual(ids(filter_tasks(TASKS, " OPEN ")), [1, 3, 5])


class NegativeTests(unittest.TestCase):
    def test_done_filter_never_contains_open(self):
        for t in filter_tasks(TASKS, "Done"):
            self.assertEqual(t["status"].lower(), "done")

    def test_open_filter_never_contains_done(self):
        for t in filter_tasks(TASKS, "open"):
            self.assertNotEqual(t["status"].lower(), "done")

    def test_unknown_status_is_rejected_not_silently_empty(self):
        with self.assertRaises(ValueError):
            filter_tasks(TASKS, "archived")

    def test_open_and_done_partition_all(self):
        self.assertEqual(sorted(ids(filter_tasks(TASKS, "open")) + ids(filter_tasks(TASKS, "done"))),
                         sorted(ids(TASKS)))


class PreservedBehaviour(unittest.TestCase):
    """Existing behaviour must not change."""

    def test_default_list_equals_baseline(self):
        self.assertEqual(list_tasks(TASKS), tracker_before.list_tasks(TASKS))

    def test_order_preserved(self):
        self.assertEqual(ids(filter_tasks(TASKS)), ids(TASKS))

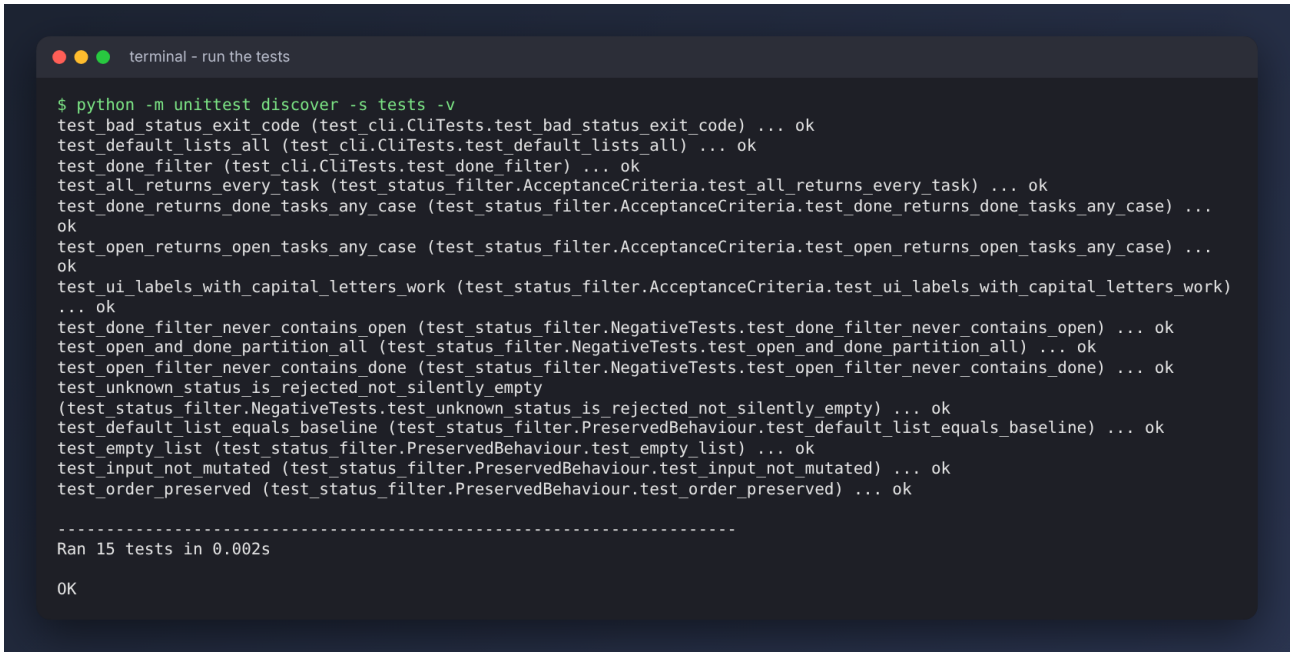
    def test_input_not_mutated(self):
        before = json.dumps(TASKS)
        filter_tasks(TASKS, "done")
        self.assertEqual(json.dumps(TASKS), before)

    def test_empty_list(self):
        self.assertEqual(filter_tasks([], "done"), [])

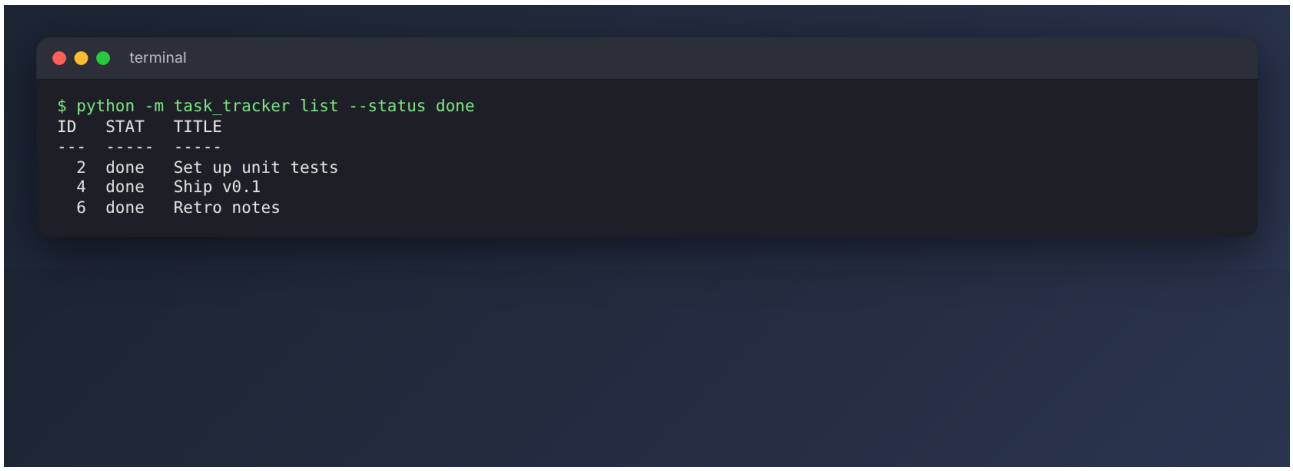

if __name__ == "__main__":
    unittest.main()
```

Test file: tests/test_status_filter.py

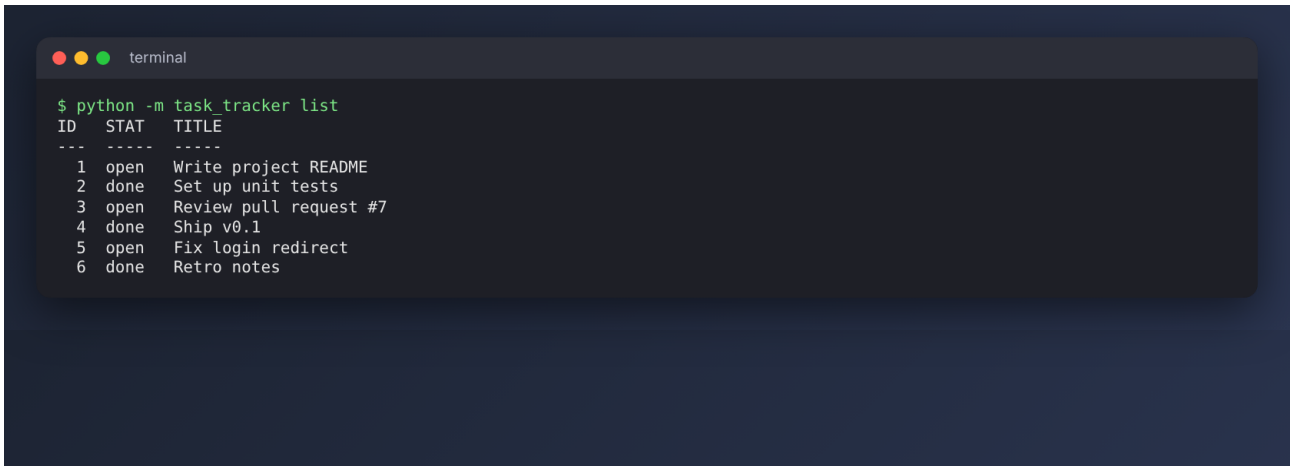
6. Demo and test screenshots



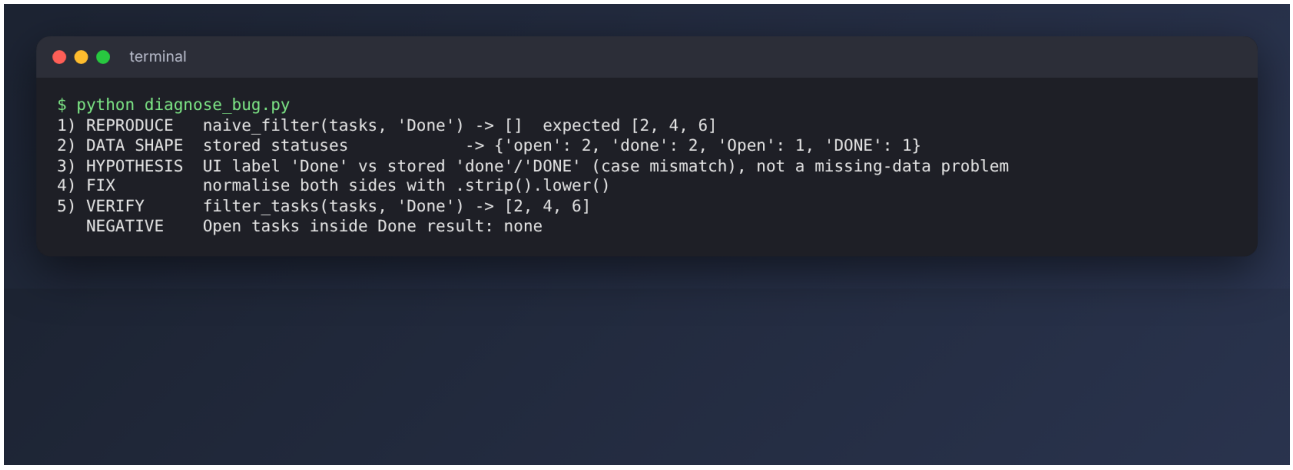
Test run (real output)



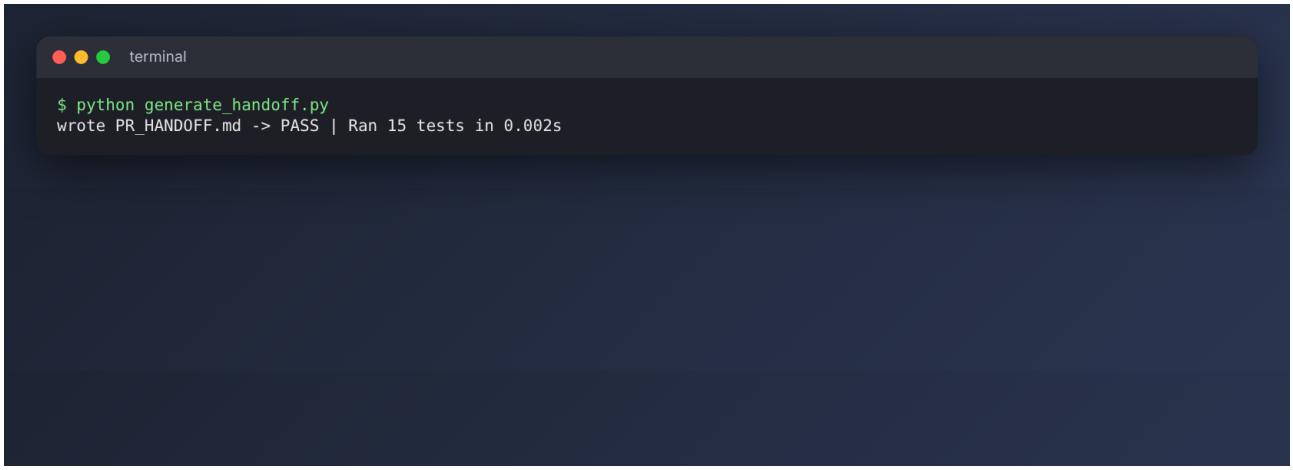
CLI: list --status done



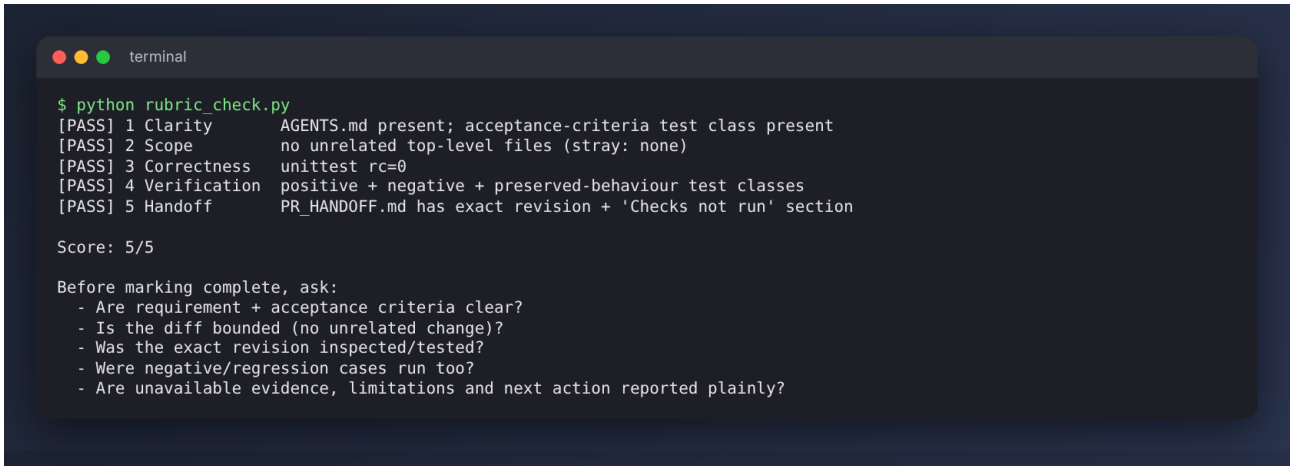
CLI: list (existing behaviour preserved)



Bug diagnosis script



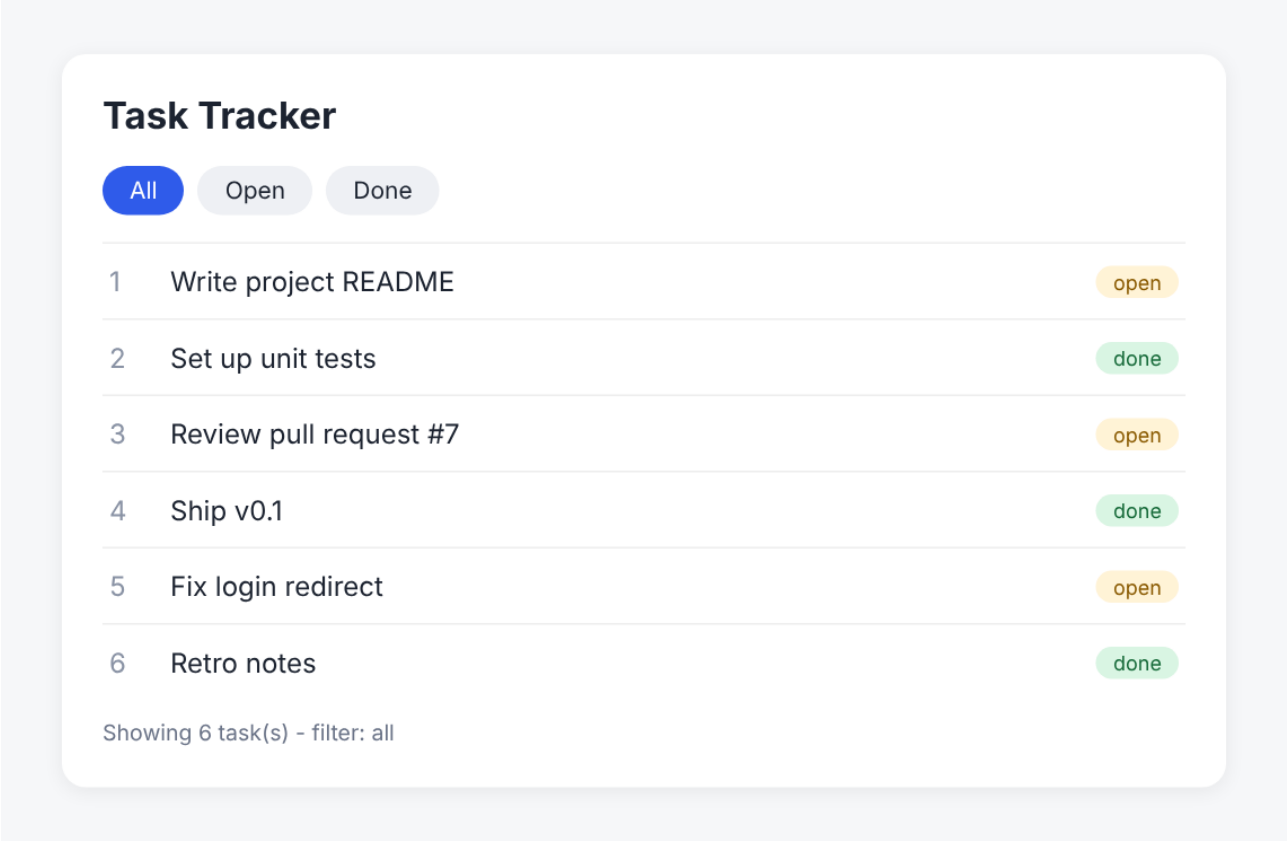
PR handoff generated from real results



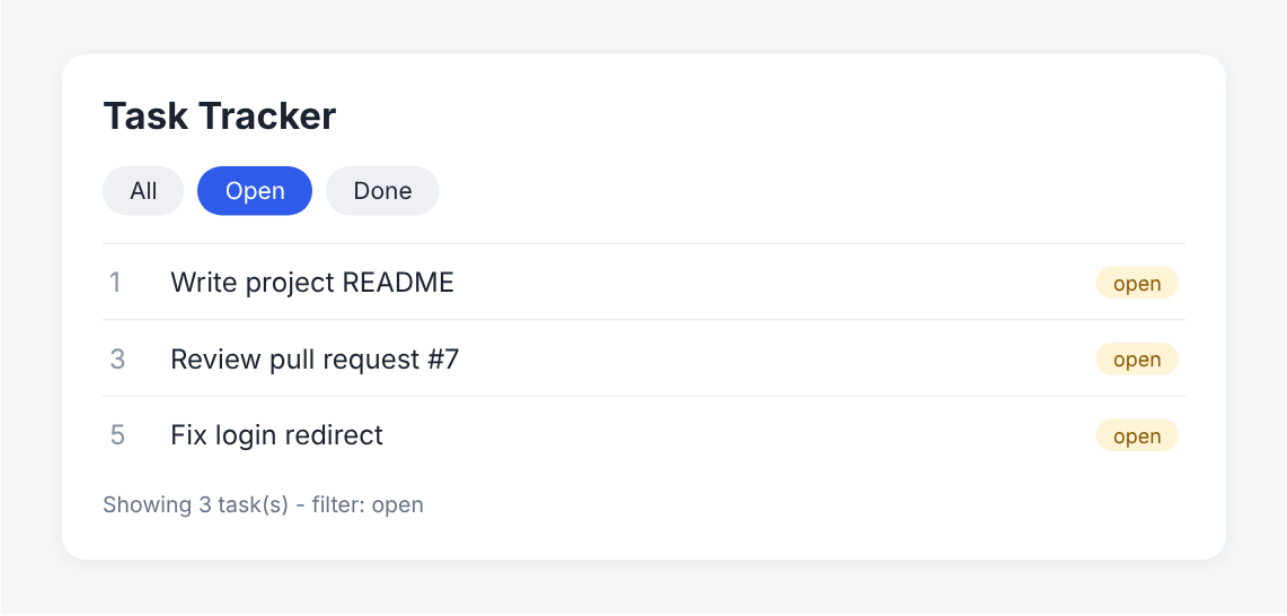
5-point rubric self-check



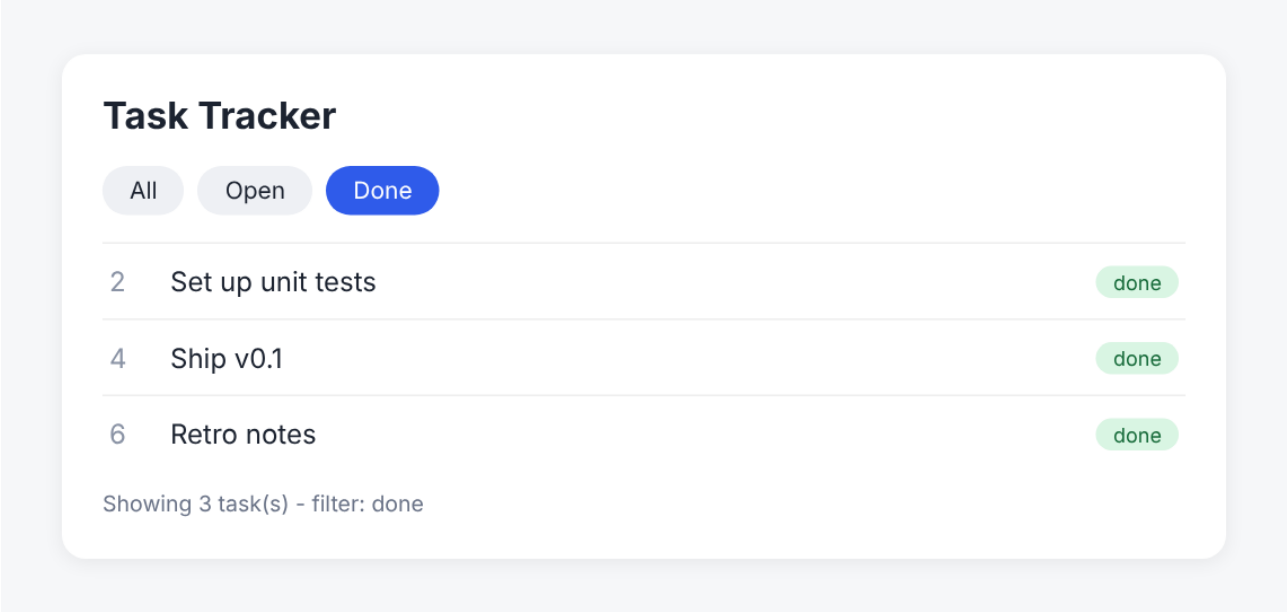
Full demo run



UI render - All



UI render - Open



UI render - Done

7. Evidence-backed report

Change	Practice Lab: Task Tracker Status Filter + PR Handoff
Revision tested	content hash `775c3c4cc5fb` of all .py files in this folder (re-run with the git commit SHA after you commit)
Checks run	`python -m unittest discover -s tests` and `python demo.py`
Results	**PASS** - Ran 15 tests in 0.002s
Checks not run	Hosted CI (runs on GitHub after push); Manual browser testing (static HTML render only)
Known limitations	Only two statuses exist; the rubric is an automated aid, not a replacement for review.
Next required action	Human review, then CI on the pull request

8. Practical takeaway

A confident developer defines scope, inspects evidence, reports uncertainty and involves a human at the right time.